

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
df=pd.read_csv("online_retail.csv")
```

```
df.head(5)
```

	InvoiceNo	StockCode	Description	Quantity	InvoiceDate	UnitPrice	CustomerID	Country
0	536365	85123A	WHITE HANGING HEART T-LIGHT HOLDER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom
1	536365	71053	WHITE METAL LANTERN	6	2010-12-01 08:26:00	3.39	17850.0	United Kingdom
2	536365	84406B	CREAM CUPID HEARTS COAT HANGER	8	2010-12-01 08:26:00	2.75	17850.0	United Kingdom
3	536365	814000	KNITTED UNION FLAG HOT WATER	6	2010-12-01 08:26:00	2.55	17850.0	United Kingdom

```
df.describe()
```

	Quantity	UnitPrice	CustomerID
count	541909.000000	541909.000000	406829.000000
mean	9.552250	4.611114	15287.690570
std	218.081158	96.759853	1713.600303
min	-80995.000000	-11062.060000	12346.000000
25%	1.000000	1.250000	13953.000000
50%	3.000000	2.080000	15152.000000
75%	10.000000	4.130000	16791.000000
max	80995.000000	38970.000000	18287.000000

```
df = df[['CustomerID', 'Quantity', 'UnitPrice']]
df['TotalPrice'] = df['Quantity'] * df['UnitPrice']
df.tail(5)
```

	CustomerID	Quantity	UnitPrice	TotalPrice
541904	12680.0	12	0.85	10.20
541905	12680.0	6	2.10	12.60
541906	12680.0	4	4.15	16.60
541907	12680.0	4	4.15	16.60
541908	12680.0	3	4.95	14.85

```
customer_data = df.groupby('CustomerID').agg({
    'Quantity': 'sum',
    'TotalPrice': 'sum'
}).reset_index()
X = customer_data[['Quantity', 'TotalPrice']]
```

```
from sklearn.cluster import KMeans
from sklearn.cluster import AgglomerativeClustering
```

```
kmeans = KMeans(n_clusters=3, random_state=0)
```

```
customer_data['Cluster'] = kmeans.fit_predict(X)
print(customer_data.head())
```

	CustomerID	Quantity	TotalPrice	Cluster
0	12346.0	0	0.00	0
1	12347.0	2458	4310.00	0
2	12348.0	2341	1797.24	0
3	12349.0	631	1757.55	0
4	12350.0	197	334.40	0

```
agg = AgglomerativeClustering(n_clusters=3)
customer_data['Agg_Cluster'] = agg.fit_predict(X)
```

```
print(customer_data.head())
```

	CustomerID	Quantity	TotalPrice	Cluster	Agg_Cluster
0	12346.0	0	0.00	0	0
1	12347.0	2458	4310.00	0	0
2	12348.0	2341	1797.24	0	0
3	12349.0	631	1757.55	0	0
4	12350.0	197	334.40	0	0

```
from sklearn.metrics import silhouette_score
```

```
kmeans_score = silhouette_score(X, customer_data['Cluster'])
agg_score = silhouette_score(X, customer_data['Agg_Cluster'])
print("KMeans Silhouette Score:", kmeans_score)
print("Agglomerative Silhouette Score:", agg_score)
```

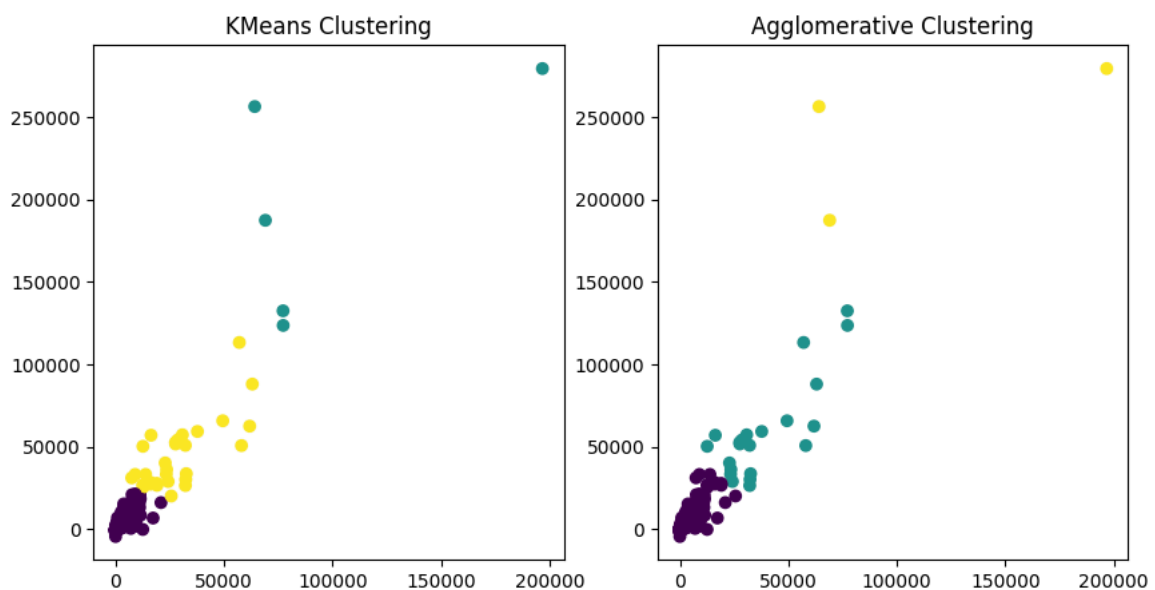
```
KMeans Silhouette Score: 0.9535126321340643
Agglomerative Silhouette Score: 0.9636717091220284
```

```
plt.figure(figsize=(10,5))

plt.subplot(1,2,1)
plt.scatter(customer_data['Quantity'],
            customer_data['TotalPrice'],
            c=customer_data['Cluster'])
plt.title("KMeans Clustering")

plt.subplot(1,2,2)
plt.scatter(customer_data['Quantity'],
            customer_data['TotalPrice'],
            c=customer_data['Agg_Cluster'])
plt.title("Agglomerative Clustering")

plt.show()
```



```
stock_df = pd.read_csv("A.csv")
stock_df.head(5)
```

	Date	Open	High	Low	Close	Adj Close	Volume
0	1999-11-18	32.546494	35.765381	28.612303	31.473534	26.929766	62546380.0
1	1999-11-19	30.713518	30.758226	28.478184	28.880545	24.711121	15234146.0
2	1999-11-22	29.551144	31.473534	28.657009	31.473534	26.929766	6577870.0
3	1999-11-23	30.400572	31.205294	28.612303	28.612303	24.481598	5975611.0
4	1999-11-24	28.701717	29.998213	28.612303	29.372318	25.131897	4843231.0

```
df.columns
```

```
Index(['CustomerID', 'Quantity', 'UnitPrice', 'TotalPrice'], dtype='object')
```

```
X = df[['Quantity', 'UnitPrice']]
y = df['TotalPrice']
```

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled,y,test_size=0.2,random_state=42)
```

```
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
```

```
xgb = XGBRegressor(n_estimators=100,learning_rate=0.1,max_depth=6,srandom_state=42)
xgb.fit(X_train, y_train)
pred_xgb = xgb.predict(X_test)
```

```
/usr/local/lib/python3.12/dist-packages/xgboost/training.py:200: UserWarning: [05:36:49] WARNING: /__w/xgboost/xgboost/src/1
Parameters: { "srandom_state" } are not used.
```

```
bst.update(dtrain, iteration=i, fobj=obj)
```

```
dt = DecisionTreeRegressor()
dt.fit(X_train, y_train)
pred_dt = dt.predict(X_test)
```

```
rf = RandomForestRegressor()
rf.fit(X_train, y_train)
pred_rf = rf.predict(X_test)
```

```
from sklearn.metrics import accuracy_score, mean_squared_error
```

```
print("XGB MSE:", mean_squared_error(y_test, pred_xgb))
print("DT MSE:", mean_squared_error(y_test, pred_dt))
print("RF MSE:", mean_squared_error(y_test, pred_rf))
```

```
XGB MSE: 322252.2744299839
DT MSE: 168630.47248542102
RF MSE: 146820.2613165849
```

```
print("XGB Accuracy:", xgb.score(X_test, y_test))
print("DT Accuracy:", dt.score(X_test, y_test))
print("RF Accuracy:", rf.score(X_test, y_test))
```

```
XGB Accuracy: 0.018463808891003408
DT Accuracy: 0.4863747293606375
RF Accuracy: 0.5528056386096382
```

```
model_scores = {
    "XG Boost": xgb.score(X_test, y_test),
    "Decision Tree": dt.score(X_test, y_test),
    "Random Forest": rf.score(X_test, y_test)
}

best_model = max(model_scores, key=model_scores.get)

print("Best Model:", best_model)
print("Best Accuracy:", model_scores[best_model])
```

```
Best Model: Random Forest
Best Accuracy: 0.5528056386096382
```

```
plt.figure(figsize=(8,5))
models = ['XGBoost', 'Decision Tree', 'Random Forest']
accuracy = [xgb.score(X_test, y_test),dt.score(X_test, y_test),rf.score(X_test, y_test)]
bars = plt.bar(models, accuracy, color='green')
plt.title('Accuracy Comparison of Models')
plt.xlabel('Models')
plt.ylabel('R² Score')
plt.show()
```

Accuracy Comparison of Models

